

Lesson 19 – Data Logging and Drive Methods Update

In this lesson you will continue to work on your branch of the RomBot2026 code.

In lesson 16 you set up manual drive methods, so for review you will get you Romi back to driving with the Xboxcontroller.

You will also learn about logging.

Important: Update your branch from Main.

Exercise 1

Task: Create a command DriveStraightDistance and add a Widget to Elastic to start the command.

Task: Create DriveWithJoystick command.

Task: Make DriveWithJoystick the default command for the romiDrivetrain subsystem.

Data Logging:

The data logging methods are in the DataLogUtil utility class.

When we want to add to the log file we will most often use the writeMessage method or the writeMessageEcho method. Both of these append data to the open log file with the latter also sending the data to the Terminal. Look at the RobotContainer code to see how logging is used. Every time the system state is changed (ie. teleop to disabled, etc.) the log file is appended to show the time that the change occurred. The method writeMessageEcho is used so the terminal is also updated to show the new state.

```
/**
 * Method called when autonomouse mode is initialized/enabled.
 */
public void autonomousInit() {
    DataLogUtil.writeMessageEcho("Auto: Mode Init ");
}
```

Change from autonomous to disabled and look at the Terminal to see these printouts. The first time you run this, a new logfile is opening with the name showing the date and time.

Exercise 2

Task: Add data logging to DriveTime in initialize().

Task: Add Widgets to Elastic to display percent setting and velocity for each motor.

(This data is sent to the SmartDashboard in RomiDriveTrain)

Task: Test and debug.

Task: Commit all code and push to remote!!

We mentioned that this data is so important to be able to analyze the robot performance for auto routines and during the match. We have to control how often we log data and how much data is logged. The writing the data to the file takes time and can use valuable system resources which can degrade the actual performance if too excessive. If we keep adding data and don't delete old files, we can run out of onboard memory space and crash the program. That is why you will see log rotation and levels of logging to control these resources. When debugging, you may want to record extra data, but during a competition match, reduce the logging to just what is essential for evaluation.